

EX No:3	DATA CLEANING

Aim :

To perform data cleaning using the Pandas library by identifying and handling missing values, duplicate records, and inconsistent data. To prepare a clean and reliable dataset for accurate data analysis and machine learning tasks.

Algorithm :

1. Import the Pandas library.
2. Load the dataset into a Pandas DataFrame.
3. Inspect the dataset for missing values, duplicate records, and incorrect data types.
4. Handle missing values by removing or filling them with appropriate values.
5. Remove duplicate records from the dataset.
6. Correct inconsistent or invalid data entries and convert data types if required.
7. Verify the cleaned dataset and save it for further analysis.

Program :

```
# Import the required libraries
import pandas as pd
import numpy as np

# Create a sample DataFrame with missing values, duplicates, and inconsistent data
df = pd.DataFrame({
    "Name": [
        "Alice",
        "Bob",
        "Alice",      # Duplicate name
        "David",
        "Nyang",
        "Frank"
    ],
    "Age": [
        25.0,
        np.nan,      # Missing age
        25.0,
        17.0,
        30.9,
        999.0        # Invalid/outlier age
    ],
    "Salary": [
        50000.0,
        60000.0,
        50000.0,
        45000.0,
        np.nan,      # Missing salary
        70000.0
    ],
    "Place": [
        "Chennai",
        "Delhi",
        "Chennai",
        np.nan,      # Missing place
        "Mumbai",
        "Delhi"
    ],
})
```

```

    "Salary": [
        50000.0,
        60000.0,
        50000.0,
        45000.0,
        np.nan,      # Missing salary
        70000.0
    ],
    "Place": [
        "Chennai",
        "Delhi",
        "Chennai",
        np.nan,      # Missing place
        "Mumbai",
        "Delhi"
    ],
    "Joined": [
        "2021",
        "2026",
        "2021",
        "2019",
        "2022",
        "2023"
    ]
}
})

# Display the DataFrame
print(df)

```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna()) #TO DROP NULL COLUMNS
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
2	Alice	25.0	50000.0	Chennai	2021
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna(how="all")) #TO DROP NULL COLUMNS WHEN ALL VALUES ARE NULL
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna(thresh=4)) #TO DROP NULL ROWS IF THERE ARE MORE THAN 4 ROWS ARE NULL
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna(subset=["Name", "Salary"])) # TO DROP ROWS IF THERE IS NULL IN NAME OR SALARY
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.duplicated()) # TO PRINT THE BOOLEAN VALUES OF DUPLICATED ROWS
```

0	False
1	False
2	True
3	False
4	False
5	False

```
dtype: bool
```

```
print(df.duplicated().sum()) #TO PRINT NUMBER OF ROWS OF DUPLICATED VALUES
```

1

```
# Display all duplicate rows in the DataFrame
print(df[df.duplicated(keep=False)])
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
2	Alice	25.0	50000.0	Chennai	2021

```
# Remove duplicate rows and store the cleaned DataFrame in a new variable
df_clean = df.drop_duplicates()
```

```
# Display the cleaned DataFrame
print(df_clean)
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
# Display the number of rows and columns after removing duplicates
print(df_clean.shape)
```

(5, 5)

```
# Remove duplicate rows based on the 'Name' and 'Age' columns
# Keep the last occurrence of each duplicate and update the original DataFrame
df.drop_duplicates(
    subset=["Name", "Age"],
    keep="last",
    inplace=True
)

# Display the updated DataFrame
print(df)
```

	Name	Age	Salary	Place	Joined
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dtypes) #to print data types of columns
```

```
Name      object
Age        float64
Salary     float64
Place      object
Joined     object
dtype: object
```

```

# Fill missing values in the 'Age' column with 0 and convert the data type to integer
df["Age"] = df["Age"].fillna(0).astype(int)

# Display the updated 'Age' column
print(df["Age"])

# Display the data types of all columns
print(df.dtypes)

```

```

1      0
2     25
3     17
4     30
5    999
Name: Age, dtype: int64
Name      object
Age      int64
Salary   float64
Place    object
Joined    object
dtype: object

```

```

# Convert the 'Joined' column from string format to datetime format
df["Joined"] = pd.to_datetime(df["Joined"], format="%Y")

# Display the converted 'Joined' column
print(df["Joined"])

# Display the data type of the 'Joined' column
print(df["Joined"].dtype)

```

```

1    2026-01-01
2    2021-01-01
3    2019-01-01
4    2022-01-01
5    2023-01-01
Name: Joined, dtype: datetime64[ns]
datetime64[ns]

```

```

: # Convert the 'Age' column to numeric values and replace invalid entries with NaN
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")

print(df["Age"])
print(df.dtypes)

```

```

1      0
2     25
3     17
4     30
5    999
Name: Age, dtype: int64
Name      object
Age      int64
Salary   float64
Place    object
Joined   datetime64[ns]
dtype: object

```

```

: #to rename columns
df.rename(
    columns={
        "Name": "Full_Name"
    },
    inplace=True
)

print(df)

```

	Full_Name	Age	Salary	Place	Joined
1	Bob	0	60000.0	Delhi	2026-01-01
2	Alice	25	50000.0	Chennai	2021-01-01
3	David	17	45000.0	NaN	2019-01-01
4	Nyang	30	NaN	Mumbai	2022-01-01
5	Frank	999	70000.0	Delhi	2023-01-01

```

df.rename(
    columns={
        "Age": "Years",
        "Salary": "Income",
        "Place": "City",
        "Joined": "Joining_Date"
    },
    inplace=True
)

print(df)

```

	Full_Name	Years	Income	City	Joining_Date
1	Bob	0	60000.0	Delhi	2026-01-01
2	Alice	25	50000.0	Chennai	2021-01-01
3	David	17	45000.0	NaN	2019-01-01
4	Nyang	30	NaN	Mumbai	2022-01-01
5	Frank	999	70000.0	Delhi	2023-01-01

Result :